COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 01

Starting as an engineer

Four expanded lessons with worked examples, practical assignments and five-question quizzes. Use the main workbook for this module’s project and additional exercises.

4 lessons · 4 demonstrations · 20 lesson questions

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Contents

Use the linked contents or PDF bookmarks to navigate.

Module 01: Starting as an engineer 3
Lesson 01.1: What a program actually does 3
Lesson 01.2: Files, terminals and reproducible execution 6
Lesson 01.3: Trace before running 9
Lesson 01.4: Units, budgets and engineering notebooks 12

Module 01: Starting as an engineer

Prerequisites: None. Basic reading, arithmetic and the ability to save a file.

Integration project: An explainable motion experiment. The full brief and review are in the Lab Workbook.

Lesson 01.1: What a program actually does

Learning outcome: Explain state, instructions, input and output using a tiny game.

The four parts of a tiny game

Imagine a maintenance robot in a training yard. It has a location, some energy and a destination. A player asks it to move right. Before writing any code, separate four things: the **input** is the request; the **state** is the stored location and energy; the **rule** decides what the request is allowed to change; the **output** tells the player what happened. A rule may reject a request. Rejection is still a defined result, not the absence of programming.

Suppose the robot starts at location 2 and receives a request to move right by 3 units. The rule produces location 5. Showing `Robot at 5` on a terminal does not create a second movement. Neither does showing the same state in a second window. This distinction is the first version of an engine boundary: simulation changes the world; presentation observes it. It will later make headless tests, replays, multiple views and dedicated servers possible within designs that preserve the boundary.

A useful rule can be written as `next state = update(current state, command)`. This is a design model, not Python syntax. Its value is that it forces us to name the information a decision needs. When a program gives an unexpected answer, ask which input, state value or rule differed from the model.

What information must persist?

A robot position must survive between commands. A message such as `Moved right` may be temporary. A movement speed is a configuration value unless the game changes it during play. A wall map is also information; even when it is constant, a collision rule depends on it. Do not call every variable state without explaining its lifetime and purpose.

Consider an inventory display reading `3 keys`. The text can be regenerated from the number 3. Storing both the number and the text as independently editable values creates a consistency problem: which is correct after a key is spent? For our first game, store the underlying facts and derive the display. Later systems sometimes cache derived values for performance, but then they must define when the cache becomes invalid.

Source code is not the screen

Source code is text written for a programming language. In this course a Python interpreter runs that text; the operating system provides services such as files and terminal access. Hardware executes the underlying instructions. An engine is another layer of software, not a special kind of processor. Keep a simple layered explanation in your notebook: game rules, reusable engine facilities, operating-system services, processor and memory.

You do not need to understand instruction decoding to begin. You do need to avoid explanations such as “the computer knew what I wanted.” A program acts according to the instructions and inputs it actually receives. A compiler or interpreter can detect some mistakes, but it does not know whether you intended a closed door or an open one.

Predict a complete transition

Make a table with columns for step, command, previous position, change and next position. Record the initial position as step zero. Do not skip a zero-distance command: its unchanged result is useful evidence that doing nothing is a legitimate case. When your final position happens to match the expected value, still check intermediate positions. Two wrong updates can cancel and produce a misleading final answer.

Our first exercise is deliberately small enough to solve on paper. A prediction is more valuable than an attractive interface at this stage. Keep the paper trace even after the program agrees with it; it becomes an independent expected result rather than an answer copied from the implementation.

Worked example

Predict before running `examples/example_01_1.py`.

```
position = 2
print("start:", position)
position = position + 3
print("after right 3:", position)
position = position - 1
print("after left 1:", position)
print("draw again:", position)
```

The first assignment establishes a fact; the first print only reports it. The next two assignments change that fact in sequence. The last print reads the same value as the preceding print, because no rule has changed the position between them. Predict four displayed values before running. Replacing the last print with `position = position + 1` would be a rule change, not a harmless drawing change.

Expected demonstration output

```
start: 2
after right 3: 5
after left 1: 4
draw again: 4
```

Practical work and pass evidence

Start on paper at x=2. Trace right 3, left 1, right 0, left 4, right 2. Include the initial state and all five following states. Mark which operations are reads and which are writes. Then run `examples/example_01_1.py` twice and compare its smaller demonstration with your prediction.

Create `workspace/01_1_trace.md`. Include the six-position trace, a plain-language update rule, and an example where two wrong movements cancel. Your pass evidence is an explanation of every transition and why drawing twice must not move twice. The optional function scaffold `labs/student/lab_01_1.py` is a return visit after Lesson 02.4; functions and loops are not prerequisites for this paper-first lesson.

Changed-case challenge

Change the initial location to -3 without changing the commands. Predict every state, not only the final value.

Companion source study

[S01] Harvard CS50 Introduction to Programming with Python; [S02] Python: An Informal Introduction. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is the difference between state and a displayed image?
2. What final position results from the five lab commands?
3. Why should drawing a character not update its position?
4. A game stores health=10 and a label reading "10". Damage changes health to 7 but not the label. Which information should normally be authoritative in this first game, and how would you prevent disagreement?
5. Two implementations finish at x=2. Is that sufficient to show they applied the five commands correctly?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 01.2: Files, terminals and reproducible execution

Learning outcome: Run a saved Python program and distinguish a shell from an interpreter.

Four objects that are easy to confuse

A **file** stores bytes. A **folder** organizes file names. A **terminal** is an interface for text interaction. A **shell** is the program interpreting shell commands in that interface. Python is another program the shell can launch. A terminal window may therefore show a shell prompt at one moment and a Python prompt at another. The visible window alone does not identify which language currently accepts your input.

A command such as `python3 examples/example_01_1.py` belongs in the shell. A statement such as `print(2 + 3)` belongs in Python code or at a Python prompt. Do not copy a prompt marker such as `$`, `PS>` or `>>>` into the command itself. The marker is notation showing where the command is entered, not part of what you type.

Locate the package, then locate the program

Extract the ZIP before executing its files. Open a terminal at the extracted `Game_Engineering_Package_02` folder. Its immediate children include `examples`, `tools`, `labs` and `capstone`. The working directory should be that folder, not its parent and not an unrelated downloads folder. A relative path such as `examples/example_01_1.py` is interpreted from the working directory.

On macOS/Linux, use `pwd` to see the directory and `ls` to inspect it. In PowerShell, use `Get-Location` and `Get-ChildItem`. To change directories, use `cd` followed by the real folder path. Quote a path that contains spaces. Do not substitute a fictional user name from someone else's example into your own filesystem.

Run `python3 --version` on macOS/Linux or `py -3 --version` on Windows. A working `python --version` that reports Python 3 can be used instead. Use the same verified launcher throughout the package. The examples target Python 3.11 or later and were executed here with Python 3.13.5; newer-version and Windows/macOS execution are not claimed as completed tests. No third-party Python package is required for the supplied runtime examples.

Read an error as evidence

A missing interpreter error comes from the shell or launcher before your script begins. A “cannot open file” message may come from Python because the interpreter ran but could not find the script. A syntax error means Python found source text but could not parse it. An exception during execution means it began running and encountered an unsupported operation or explicit failure.

These are different layers. Editing the movement formula cannot repair a misspelled filename. Reinstalling Python is not the first response to a wrong output value. Identify the layer, reproduce the failure once and change the smallest relevant thing. In this lesson, a deliberately misspelled example filename is a safe failure experiment because it does not modify files.

Record an executable recipe

A reproducibility note should name the source file or revision, interpreter version, working directory, exact command, inputs, expected output and actual output. A screenshot can supplement the record, but plain text is easier to search and copy. Avoid recording private paths or identifiers in a public portfolio without review.

Run `tools/environment_check.py` for a read-only report. It does not install software or change settings. A missing Git installation is not a blocker until the Git lesson. When software installation is needed, follow the official setup documentation and the device owner's approved process. Nothing in this volume requires administrator commands, changing security settings, a paid cloud account or an engine installation.

Worked example

Predict before running `examples/example_01_2.py`.

```
from pathlib import Path
import sys

print("python:", sys.version.split()[0])
print("script name:", Path(__file__).name)
print("working directory name:", Path.cwd().name)
print("script exists:", Path(__file__).is_file())
```

`__file__` identifies this script's file. `Path.cwd()` asks for the process working directory; it is not necessarily the directory containing the script. The two may coincide, but this example intentionally reports them separately. Version and directory-name output are environment-dependent, so compare their meaning rather than expecting another learner's exact text.

Expected demonstration output

```
python: [your actual Python version]
script name: example_01_2.py
working directory name: [your current folder name]
script exists: True
```

Practical work and pass evidence

Run `tools/environment_check.py` and save the output with your actual command in `workspace/01_2_environment.md`. Run the movement example, then deliberately change only its filename in the command to a name that does not exist. Record the resulting error, identify which program emitted it and repair the command.

Repeat the valid command from the package root. Your evidence must distinguish a script path from the working directory and explain how a relative filename is resolved. This is an evidence-based lab: no Python function starter is assigned and the automatic lab checker does not certify environment understanding.

Changed-case challenge

Run the example using its absolute path from a different working directory. Predict which reported field changes before executing.

Companion source study

[S02] Python: An Informal Introduction; [S03] Python Setup and Usage. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What does a relative filename depend on?
2. Is cd a Python statement in an ordinary script?
3. What three facts help another person reproduce a run?
4. You are in the package root, but type `python3 example_01_1.py`. The file is inside examples/. What is the smallest likely repair?
5. A saved edit produces no change in output. Give two checks before reinstalling anything.

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 01.3: Trace before running

Learning outcome: Predict a program by following assignments in order.

Assignment is an event, not a permanent equation

In mathematics, an equation often states a relationship to be solved or maintained. A Python assignment performs an operation at a particular moment. The right side is evaluated using the values available then, and the resulting object is bound to the name on the left. Later assignments do not automatically rerun earlier expressions.

With `health = 10` followed by `after_hit = health - 3`, the second name refers to the result 7. Setting `health = 20` afterward does not change `after_hit` to 17. That would require another evaluation. A display formula recalculated each frame is a separate operation; it is not a magical property of assignment.

Trace one statement at a time

Use a row after every statement that changes a name. Before a name has been assigned, mark it as “not yet defined” rather than guessing zero. A program that reads such a local name may fail; names are not automatically initialized to whatever would make the algorithm work.

For each row, write the old values needed by the right side, evaluate the expression, then update the left side. This discipline prevents a common mistake: mentally using a future value too early. It also shows why changing statement order can change behavior. “These are the same lines” is not a correctness argument when they run in a different sequence.

The overwritten-value problem

Suppose two players have scores 3 and 8 and you want to exchange them. Writing `left = right` first makes `left` refer to 8. A later `right = left` reads that new 8, not the original 3. The old left value is no longer accessible through either name. The algorithm lost information before it used it.

A temporary name preserves the old value until both assignments are complete. Python also supports multiple assignment such as `left, right = right, left`; the right-hand values are evaluated before the new bindings are installed. Use the longer temporary-variable version first so that you can explain what the shorter version accomplishes. A concise expression is useful only when its semantics are understood.

Names and values are not always independent copies

For the integer examples here, rebinding one name does not modify the immutable integer referred to by another name. Do not generalize that to every object. Later, two names can refer to the same mutable list; changing the list through one name is then observable through the other. The right mental model is “names refer to objects,” not “every assignment makes a deep copy.” Lesson 03.1 develops the distinction with concrete collections.

This limitation matters even now. A trace table should record values and, when relevant, whether two names refer to the same object. For now use integers and strings; the goal is to master sequencing before adding shared mutable structures.

Predictions that can be wrong are useful

Write the expected output before you run the program. When the result disagrees, locate the first disagreeing row rather than only the last line. Then state a specific correction to your mental model. “Python is weird” does not identify a reusable lesson; “I expected an earlier expression to reevaluate” does.

Build a habit of separating a prediction from an observation in your notebook. Do not edit the prediction after seeing the result. The difference is the evidence showing what you learned. Over time this record will make recurring assumptions visible and make debugging more systematic.

Worked example

Predict before running `examples/example_01_3.py`.

```
left = 3
right = 8
saved_left = left
left = right
right = saved_left
print("swapped:", left, right)

x = 4
y = x + 2
x = 10
print("later values:", x, y)
```

`saved_left` keeps 3 accessible while `left` is rebound to 8. `right` then receives the preserved value. In the second sequence, `y` is calculated when `x` is 4. The assignment of 10 to `x` does not execute the `y` expression again. Write both traces before running the file.

Expected demonstration output

```
swapped: 8 3
later values: 10 6
```

Practical work and pass evidence

Create three four-statement assignment sequences in `workspace/01_3_trace.md`. Include an incorrect swap, the repaired swap and a sequence in which a derived value does not update automatically. For each, predict every binding and final printed output before checking with Python.

The optional starter `labs/student/lab_01_3.py` packages the swap in a function; return to it after Lesson 02.4. Pass now by explaining where the incorrect swap loses the old value and by solving a fresh sequence without running it first. Do not count a memorized one-line swap as sufficient explanation.

Changed-case challenge

Use starting values -5 and 0 in your repaired swap. Then make a two-variable example whose behavior changes when two assignments are reversed.

Companion source study

[S02] Python: An Informal Introduction; [S04] Python Built-in Types; [S09] Python: Classes. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. After a = 3; b = a; a = 8, what is b for these integers?
2. Why does x = y; y = x fail to swap two different values?
3. What makes prediction useful when a run disagrees?
4. Trace `a=2; b=a*4; a=b-1; b=a+2`. What are a and b at the end?
5. Why should you preserve your incorrect prediction instead of rewriting it to match the output?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 01.4: Units, budgets and engineering notebooks

Learning outcome: Use units and simple arithmetic to check game calculations.

Units make arithmetic interpretable

The expression `6 * 0.25` produces a number, but its meaning depends on what the inputs represent. Six units per second multiplied by one quarter of a second gives 1.5 units. Seconds cancel. Six pixels per frame multiplied by seconds does not have the same meaning. Naming variables `speed_units_per_second` and `dt_seconds` keeps that distinction visible.

Write the relationship before substituting numbers: displacement equals speed multiplied by elapsed time. Then state assumptions. Here speed is constant during the update and the model is one-dimensional. We are not yet modeling acceleration, collision or curved motion. A correct calculation under a simple model should not be described as a complete physical simulation.

Convert at the boundary

One second is 1,000 milliseconds. When a timing source reports 250 milliseconds, convert it to 0.25 seconds before using a speed expressed per second. Multiplying 6 by 250 would produce a displacement 1,000 times too large. Adding a position to a time is a different failure: the units do not describe a meaningful sum at all.

Prefer one internal unit convention. For this volume, elapsed time is in seconds and positions use explicitly named game units. Convert input into that convention once and format output separately. Do not scatter unexplained factors of 1,000 through unrelated functions; each is another place to invert a conversion accidentally.

A frame interval is a budget, not a measurement

For a target rate of f frames per second, a evenly spaced frame interval is `1/f` seconds or `1000/f` milliseconds. At 60 frames per second that is approximately 16.67 milliseconds. It is a planning interval, not evidence that a particular computer can draw a particular scene at that rate. A 20-millisecond operation alone cannot fit inside a 16.67-millisecond serial budget.

Be careful when adding work times. A first serial model might add input, simulation and drawing durations, but later CPU/GPU overlap requires dependency reasoning. We will not infer actual frame performance from this arithmetic. For now, the exercise teaches whether a stated target and a stated cost are even compatible under the explicitly chosen serial model.

Zero, sign and non-finite values

A negative velocity can legitimately mean movement left. A negative elapsed interval is excluded by this lesson's contract because the update is not a time-reversal operation. A zero interval is allowed and produces no movement. A target frame rate of zero cannot be converted to an ordinary finite frame interval by division.

Computers also have representations such as positive infinity and not-a-number. Later code uses `math.isfinite` to reject them at the boundary of calculations that require finite values. This does not replace unit reasoning: 250 milliseconds mislabeled as seconds is perfectly finite but wrong. Validation must match both representation and meaning.

Keep hypotheses separate from results

Use three headings in a laboratory note: prediction, observation and explanation. “I predict 15 units because 6 units/second times 2.5 seconds equals 15 units” is a prediction. “The program printed 15000” is an observation. “The input remained in milliseconds” is an explanation to test, not a fact established by the surprising output alone.

When recording measured performance later, preserve the workload, environment, trial method and raw samples. Here, record the arithmetic and chosen assumptions. Do not invent an execution-time result to fill a table. An unanswered measurement question is more useful than a precise-looking fictional number.

Worked example

Predict before running `examples/example_01_4.py`.

```
speed_units_per_second = 6.0
dt_milliseconds = 250.0
dt_seconds = dt_milliseconds / 1000.0
travel_units = speed_units_per_second * dt_seconds
print("travel units:", travel_units)

frames_per_second = 120
interval_ms = 1000.0 / frames_per_second
print("target interval ms:", round(interval_ms, 3))
```

The conversion occurs before the movement calculation. The frame result is rounded only for presentation; that does not improve the underlying timing behavior of a machine. Explain why multiplying milliseconds by 1,000 instead of dividing would create a second, much larger error. The example measures no hardware performance.

Expected demonstration output

```
travel units: 1.5
target interval ms: 8.333
```

Practical work and pass evidence

Calculate all six combinations of speeds 2, 6 and 10 units/second with intervals 0.1 and 0.5 seconds. Then calculate the frame intervals for 30, 60 and 120 frames/second. Show units and identify the chosen targets. Store the table in `workspace/01_4_units.md`.

After functions are introduced, implement the `displacement` and `frame_interval_ms` starter contracts. Their input domain is finite numeric values, with nonnegative elapsed time and positive frame rate; finite overflow results must also be rejected. These programmatic checks supplement rather than replace the written unit analysis.

Changed-case challenge

A movement program receives 16 milliseconds and a speed of 2.5 meters/second. Derive the displacement and compare it with the mistaken calculation using 16 seconds.

Companion source study

[S02] Python: An Informal Introduction; [S05] Python: Floating-Point Arithmetic; [S14] Python math library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. How far does 8 units/second travel in 0.125 seconds?
2. What is the interval for 120 frames/second?
3. Why record hardware and workload with a timing result?
4. A serial frame has 2 ms input work, 5 ms simulation and 11 ms drawing. Does that model fit a 60-frame/s interval?
5. Why can a finite-value check accept a calculation that is wrong by a factor of 1,000?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.